day04am

Tree Basics

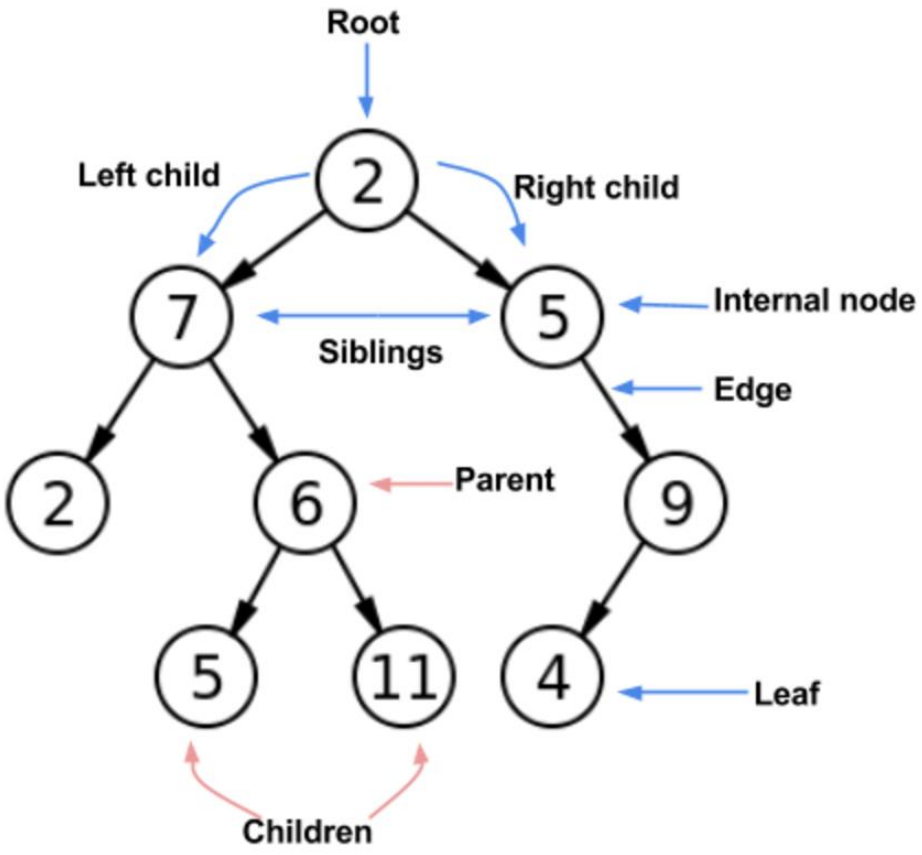
Kittipitch Kuptavanich

2026-03-30

Grader

- **C2P77 – BT-Ops**

Trees - Overviews



- **Node เชื่อมกันด้วย Edge.**
- **Node พิเศษที่เรียกว่า Root โดยทั่วไปจะอยู่บนสุด**
- **มีเส้นทางเดียวจาก Root ไปยัง Node ใด ๆ**
- **Edges แสดงความสัมพันธ์ แบบ parent/child (like family tree)**
- **ทุก Node ยกเว้น Root มี 1 Parent**
- **Nodes ที่มี Parent เดียวกันเรียกว่า Siblings**
- **Nodes ที่ไม่มีลูกเรียกว่า Leaf/Leaves ที่เหลือเรียก Internal Nodes**

Tree Terminology

- **Size คือจำนวน Node**
- **Level (Depth) ของ Node = 1 + Level ของ Parent**
- **Level (Depth) ของ Root = 0**
- **Height = จำนวน Level (รูปทางขวา Height = 4)**
- **มี Node เดียวก็เป็น Tree ได้**
  - **1 node, 0 edges**
- **Q: Tree has n nodes. How many edges?**
  - **A: _____**

```
      2
     / \
    7   5
   / \   \
  8   6   9
     / \   \
    5  11   4
```

Binary Tree

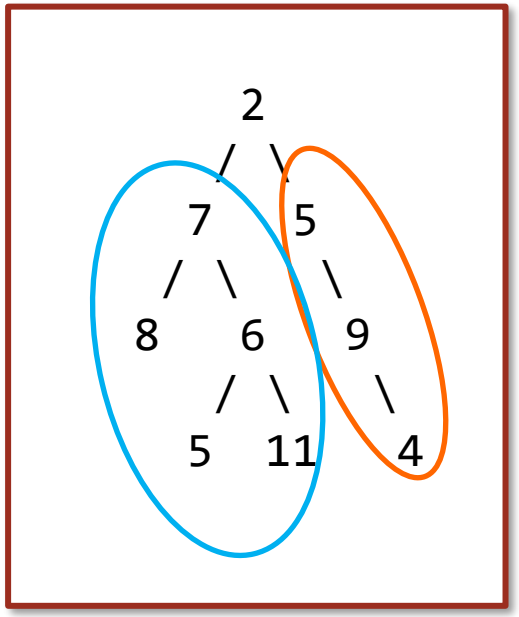
- ทุก Node มี Children ได้มากที่สุดแค่ 2.
- ใน Binary Tree เราจะแยก ระหว่าง LEFT child และ RIGHT child.
- BTS (Binary Search Tree) เป็น Special Class of Binary Tree (คาบบ่าย)

```
// E.g., represent tree storing ints in C++ this way:
struct BinTree {
  int value;
  int id;  // can also keep unique keep id in case there are duplicated value
  BinTree *left;
  BinTree *right;
};
```

Trees and Recursion

Trees และ recursion มีความเชื่อมโยงกันอย่างมาก เราสามารถพูดได้ว่า โดยนิยามแบบ Recursive แล้ว

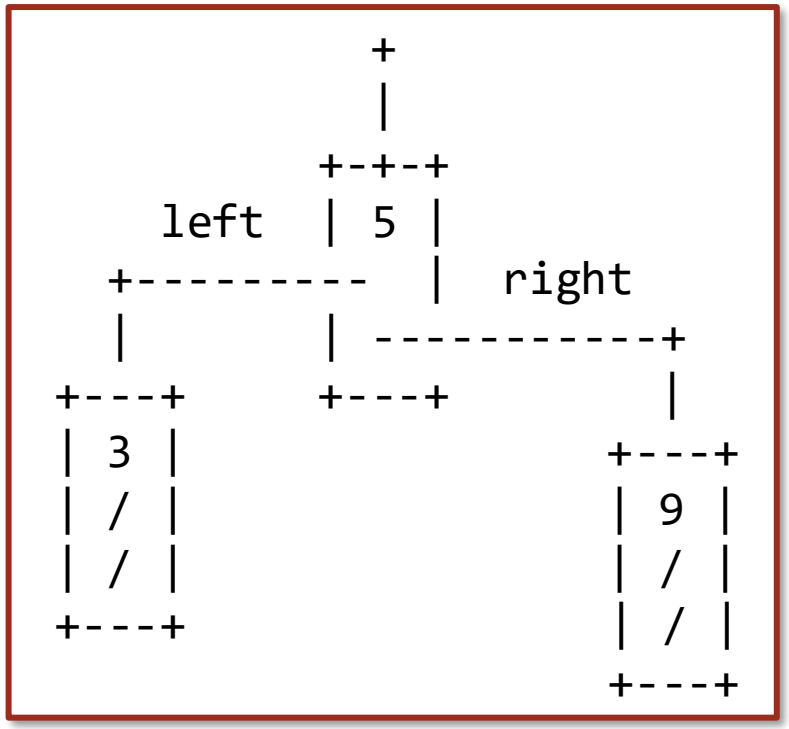
- **Binary Tree มีสองกรณี**
  - **(A) เป็น Tree ว่าง**
  - **(B) เป็น Root node ที่มี Children เป็น binary tree ซ้าย และ binary tree ขวา**



Leaf Nodes

- จาก struct definition, binary tree พิจารณารูปขวา
- คำถาม: เราต้องการนับ Pointer ที่เป็น NULL เป็น Leaf Node หรือไม่
  - ตอบ: บางทีก็นับ แล้วแต่ Definition
    - ถ้านับ Null Pointer เป็น Leaf ทุก Node จะมี 2 children (เรียกว่า "extended binary tree")
- สมมติว่าเรานับ NULL เป็น Leaf Node จำนวน Leaf Node (Null Pointer) เท่ากับ L เราจะมี Internal Node กี่ Node (รูปทางขวามี 4 Leaves)
  - ตอบ: L-1

```
struct BinTree {
  int value;
  BinTree *left;
  BinTree *right;
};
```



https://en.wikipedia.org/wiki/Binary_tree

Special Types of Binary Trees

Full Binary Tree • ทุก Node มี 2 หรือ 0 children	1 / \ 2 3 / \ 6 7
Complete Binary Tree • ทุก Level เว้น Level สุดท้าย มีจำนวน Node มากที่สุดที่เป็นไปได้ และ • Level สุดท้าย อาจะมี Node ไม่ครบแต่ Node ที่มีต้องไล่จากซ้ายไปขวาเริ่มจากซ้ายสุดที่เป็นไปได้ และห้ามแหว่ง	1 / \ 2 3 / 4 1 / \ 2 3 / \ 4 5
Perfect Binary Tree • มีจำนวน Node ที่มากที่สุดที่เป็นไปได้ใน height นั้น ๆ = 2^h – 1	1 / \ 2 3 / \ / \ 4 5 6 7

https://en.wikipedia.org/wiki/Binary_tree

Special Types of Binary Trees

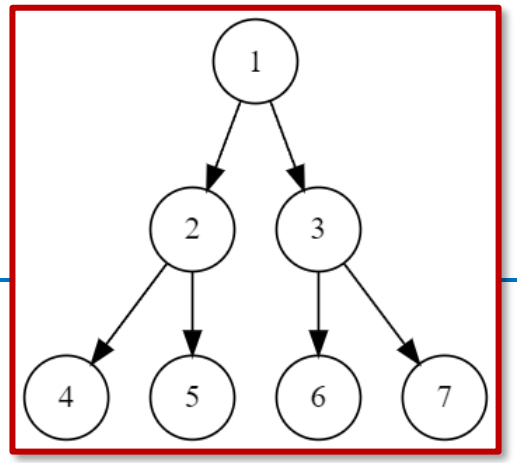
	Full	Complete
1 / \ 2 3 / \ / 4 5 6		
1 / \ 2 3 / \ \ 4 5 7		

	Full	Complete
1 / \ 2 3 / \ 6 7		
1 / \ 2 3 / 4		

Trees and Recursion [2]

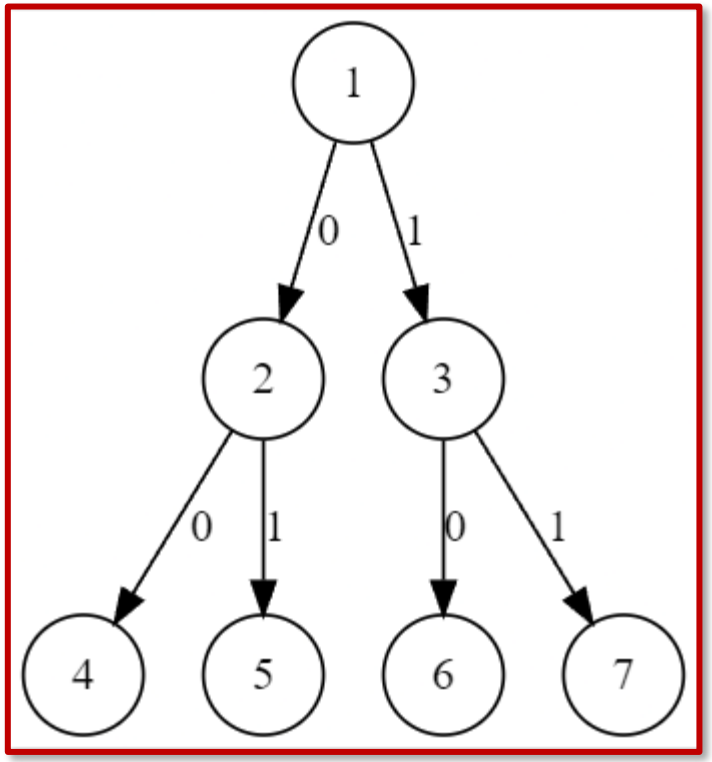
- สมมติเราต้องการเขียนฟังก์ชันเพื่อสร้าง **Perfect Binary Tree** ที่มี **Height =** h

```
BinTree * makePerfectTree(int h, int &v)
{
    BinTree * root;
    if (h == 0) return NULL;
    root = new BinTree;
    root->value = ++v;
    root->left = makePerfectTree(h - 1, v);
    root->right = makePerfectTree(h - 1, v);
    return root;
}
```



Path to Node in Binary Tree

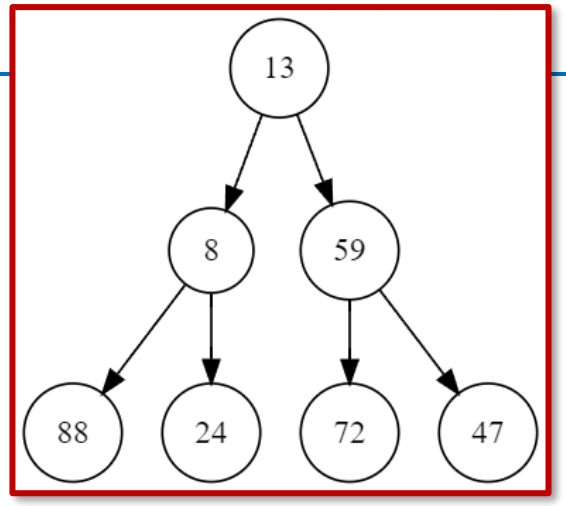
- พิจารณา Perfect Tree ที่ Label Root เป็น 1 และ Node อื่น ๆ ตามลำดับโดยเรียง ตาม Level และ ซ้ายไปขวา
  - หาก Label Edge
    - ที่เชื่อมไป Left Child = 0
    - ที่เชื่อมไป Right Child = 0
  - สังเกตความสัมพันธ์ของ Path กับเลขฐานสอง
    - Path ที่จะไปหา Node 4 จาก Root คือ 00
    - เลขฐานสองของ 4 คือ 100
  - เลขฐาน 2 ของ Node นั้น ๆ คือ '1' ตาม ด้วย Path
    - เช่น 7 คิดเป็นเลขฐาน 2 ได้ 111 ดังนั้นมี Path = 1 1 จาก Root หรือ Traverse ไปทางขวาสองครั้ง



Tree Height

- **Another recursion**

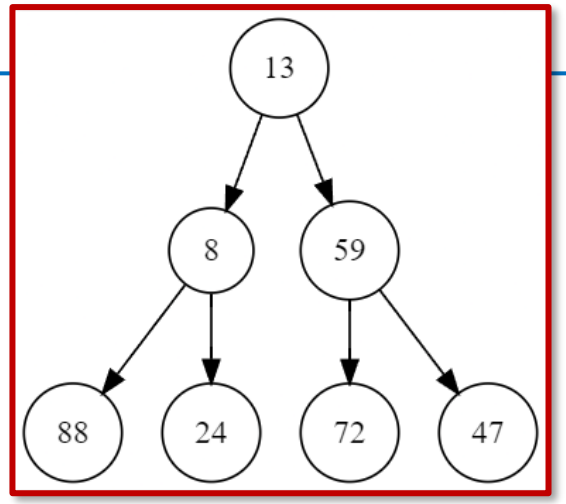
```
int treeHeight(BinTree *root)
{
    if (!root)
        return 0;

    return 1 + max(treeHeight(root->left),
                   treeHeight(root->right));

}
```



Finding a Node

- **Also easy to do w/ recursion**

```
BinTree* findNode(BinTree* root, int key)
{
    if (!root || root->value == key)
        return root;

    // Search in left subtree
    BinTree* leftResult = findNode(root->left, key);
    if (leftResult)
        return leftResult;

    // Search in right subtree
    return findNode(root->right, key);
}
```



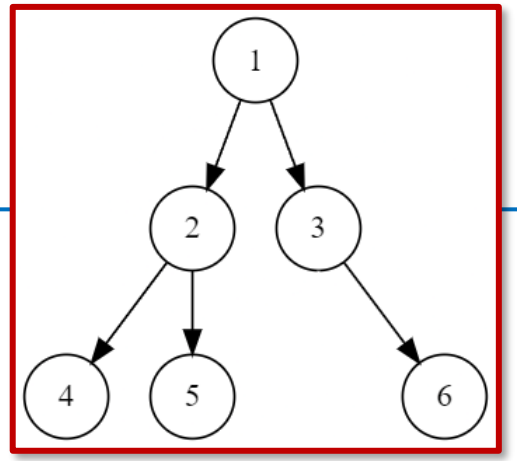
ถ้าต้องการหา **Sibling Node** หรือ **Parent Node** เมื่อกำหนด **key** ให้ จะทำอย่างไร?

Tree Printing

- **A lot easier to print it side-way for debugging.**

```
33 void printTree(BinTree* root, int level = 0, char branch = '-')
34 {
35     if (root == nullptr)
36         return;
37
38     printTree(root->right, level + 1, '/');
39     for (int i = 0; i < level; i++)
40         cout << "    ";
41     cout << branch << root->value << endl;
43     printTree(root->left, level + 1, '\\');
44 }
45
```

```
        /6
    /3
-1
        /5
    \2
        \4
```

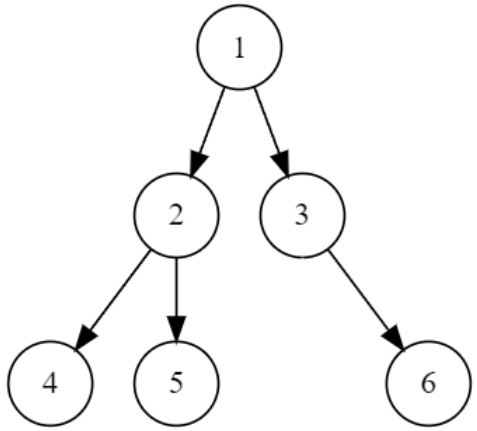


Tree Representation

กรณีที่การ implement Tree Traversal ไม่แสดง Null Children เราจำเป็นจะต้องใช้มากกว่า 1 Traversal output ในการนำมาสร้าง Tree ที่ถูกต้อง

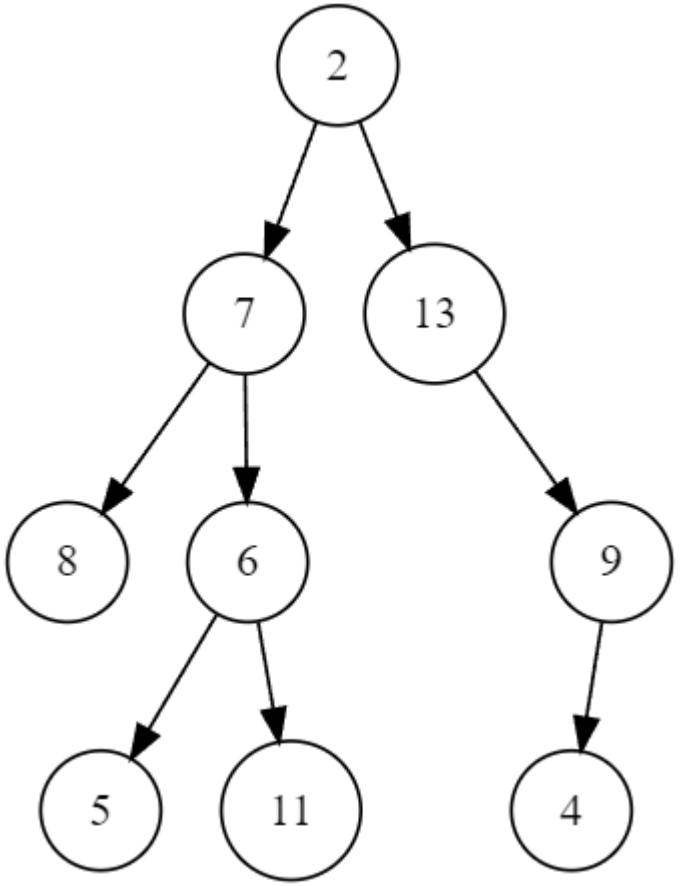
How would you represent a tree as text? Here we use # (or -1) for a null child of internal nodes

- **Level Order Traversal**
  - แสดงค่าเรียงตาม Level จากซ้ายไปขวา
  - **1 2 3 4 5 # 6**
- **Pre-order Traversal**
  - แสดงค่า Parent Node, Left subtree และ Right subtree ตามลำดับ
  - **1 2 4 5 3 # 6**



- **In-order Traversal**
  - แสดงค่า Left subtree, Parent Node และ Right subtree ตามลำดับ
  - **4 2 5 1 # 3 6**
- **Post-order Traversal**
  - แสดงค่า Left subtree, Right subtree และ Parent Node ตามลำดับ
  - **4 5 2 # 6 3 1**

Exercise



- **Pre-order**

- **In-order**

- **Post-order**

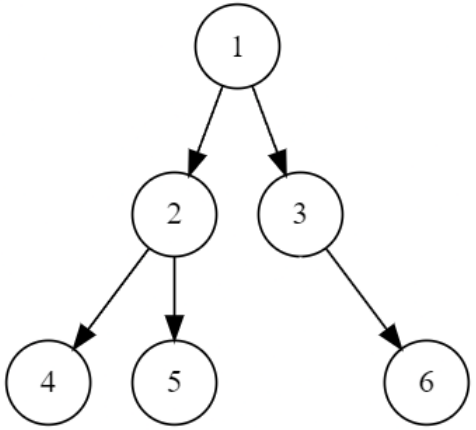
- **Level order**

Create Node

```
// Function to create a new binary tree node
BinTree* createNode(int value)
{
    BinTree* newNode = new BinTree();
    if (!newNode) {
        cout << "Memory allocation error!" << endl;
        exit(EXIT_FAILURE);
    }
    newNode->value = value;
    newNode->left = nullptr;
    newNode->right = nullptr;
    return newNode;
}

```

Manual Tree Construction

```
int main()
{
    // Manually constructing binary tree
    int arr[] = {1, 2, 3, 4, 5, -1, 6};


    // Create root node
    BinTree* root = createNode(arr[0]);

    // Create child nodes
    root->left = createNode(arr[1]);
    root->right = createNode(arr[2]);

    root->left->left = createNode(arr[3]);
    root->left->right = createNode(arr[4]);

    root->right->right = createNode(arr[6]);

    return 0;
}
```

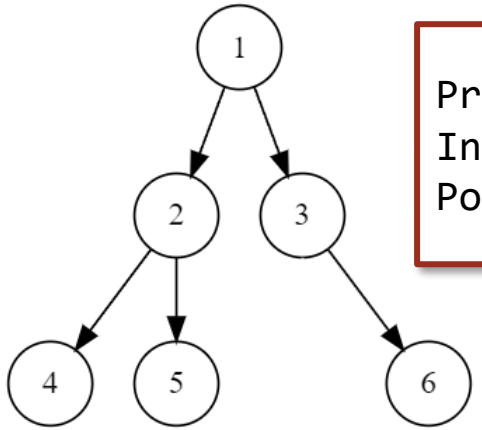


Tree Traversal

ไม่แสดง null children ทำให้ไม่สามารถ สร้าง Tree ได้ถ้าดูจาก Traversal เพียงประเภทเดียว <u>ให้ นร แก้ไขให้แสดง null children ตาม slide หน้า 10 (เฉพาะกรณีไม่ใช่ leaf node)</u>

```
// pre-order traversal
void preorderTraversal(BinTree* root)
{
    if (root) {
        cout << root->value << " ";
        preorderTraversal(root->left);
        preorderTraversal(root->right);
    }
}

// in-order traversal
void inorderTraversal(BinTree* root)
{
    if (root) {
        inorderTraversal(root->left);
        cout << root->value << " ";
        inorderTraversal(root->right);
    }
}
```

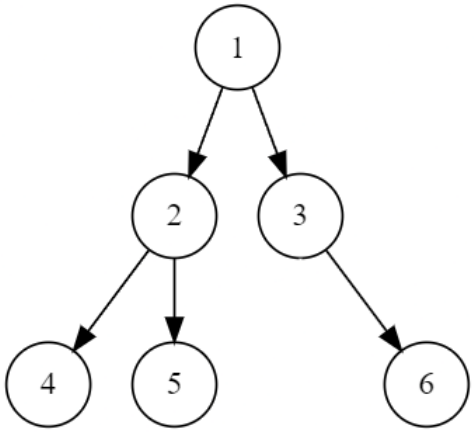
```
// post-order traversal
void postorderTraversal(BinTree* root)
{
    if (root) {
        postorderTraversal(root->left);
        postorderTraversal(root->right);
        cout << root->value << " ";
    }
}
```




```
Pre-order: 1 2 4 5 3 6
In-order: 4 2 5 1 3 6
Post-order: 4 5 2 6 3 1
```

Array Representation of a Binary Tree

- **Root: อยู่ที่ index 0.**
- **Node ที่อยู่ที่ index 'i',**
  - **Left Child: อยู่ที่ index '(2 * i) + 1'.**
  - **Right Child: อยู่ที่ index '(2 * i) + 2'.**
- **Null Children: -1 หรือ null เครื่องหมายอื่น ๆ ตามแต่จะนิยาม**
  - **รูปทางขวาจะได้ว่า Array คือ [1, 2, 3, 4, 5, -1, 6]**
- **สรุปคือเหมือน Level Order**

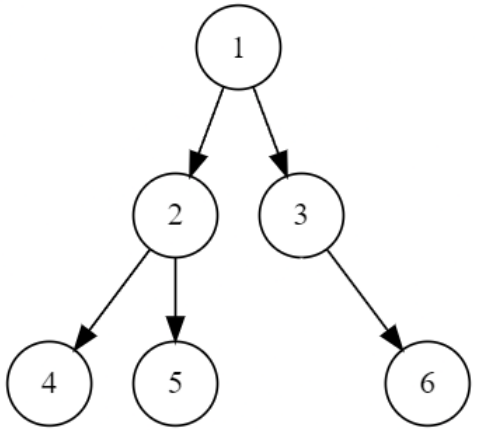


Creating a Tree from Level Order Representation

```
 5 const int NULL_VALUE = -1;
..
29  BinTree* constructTree(std::vector<int>& arr)
30 {
31     if (arr.empty() || arr[0] == -1)
32         return nullptr;
33
34     map<int, BinTree*> myMap;
35     int leftIdx;
36     int rightIdx;
37     for (int i = arr.size() - 1; i >= 0; i--) {
38         if (arr[i] != -1) {
39             myMap[i] = createNode(arr[i]);
40             leftIdx = 2 * i + 1;
41             rightIdx = 2 * i + 2;
42             if (leftIdx < arr.size() &&
43                     myMap.find(leftIdx) !=
44                     myMap.end()) {
45                 myMap[i]->left =
46                     myMap[leftIdx];
47             }
```

```
48             if (rightIdx < arr.size() &&
49                     myMap.find(rightIdx) !=
50                     myMap.end()) {
51                 myMap[i]->right =
52                     myMap[rightIdx];
53             }
54         }
55     }
56
57     return myMap[0];
58 }
```

```
95 int main()
96 {
97     vector<int> arr = {1, 2, 3, 4, 5,
98                        NULL_VALUE, 6};
99     BinTree* root = constructTree(arr);
```

Deleting a Node

- ถ้าเป็น Leaf Node
  - ทำได้เลยแก้ Pointer ของ Parent ให้ชี้ไปที่ NULL แล้ว free()
- ถ้าเป็น Internal Node
  - อาจจะต้อง Recursively ดึงค่าของ Child Node ด้านใดด้านหนึ่งมาแทนที่จนเหลือ Leaf Node ล่างสุดแล้วค่อยลบ ขึ้นกะ Algorithm และประเภท Tree